Universal Ultra VGA Engine Mod Specification

Revision 0.4

the 5th of April, 2026

UVEMod v0.3

UVEPaths v0.4

instdata v0.2

TABLE OF CONTENTS

§1 Introduction 3
§1.1 Terminology 3
§2 Mod format 4
§2.1 UVEMod signature 4
§2.2 Primary header 4
§2.3 Header chunks 4
§2.3.1 Metadata chunk 6
§2.4 Entries 8
§2.5 Combining several mods 9
§2.6 Backward compatibility with legacy formats 9
§3 File paths table format 10
§3.1 UVEPaths signature 10
§3.2 Primary header 10
§3.3 Header chunks 10
§3.3.1 Metadata chunk 11
§3.4 Entries 12
§3.5 Backward compatibility with legacy formats 12
§4 Installed mods data (“instdata”) 13
§5 file Example 14
§6 Contributors 15
§7 Changes between revisions 16
2026-04-05 Rev 0.4 16
2026-03-02 Rev 0.3 16
2026-03-02 Rev 0.2 16
2025-10-19 Rev 0.1 17

§1 INTRODUCTION

This format specification has been made with an intent to unify the UVE modding toolset, so every feature could be covered by the format structure compatible with different tools. Its primary goal is to replace the current CIUMod format, first introduced in IAMT/IAMI and later adopted as the common denominator for other tools to make mod installation simpler.

§1.1 Terminology

byte, u8, s8 — an integer of 8 bits, unsigned or signed.
u16, s16 — an integer of 2 bytes, 16 bits of data.
u32, s32 — an integer of 4 bytes.
u64, s64 — an integer of 8 bytes.
u8[x] — an array of u8 values. May be used as a filler to indicate ‘x’ bytes of data.
‘\0’ — a byte with a value of 0 in a text string.
FourCC — ‘four-character code’, a u32 value used to identify the format or a chunk of the file. Generally, it can be translated to a readable ASCII sequence. Also known as ‘magic value’ or ‘format identifier’.
Values with a 0x prefix are hexadecimal: 0x12 → 18.
Values with a 0b prefix are binary: 0b00000010 → 2.

§2 MOD FORMAT

Every value is stored in a low-endian byte order: a u16 value of 0xABCD is stored as `CD AB` in the file. All ASCII sequences are stored as-is, in a readable form: ‘abcd’ is `61 62 63 64`, or `0x64636261` in hexadecimal form.

§2.1 UVEMod signature

The first 8 bytes of an UVE mod file always contain the following byte array:

Listing 2.1.1 — UVE mod header signature

```
55 56 45 4D 6F 64 00 00
```

This translates to an ASCII sequence ‘`UVEMod\0\0`’, or an u64 value of `0x646F4D455655 (110429200537173)`. If the signature matches, the app should be able to safely assume that this is a compatible mod format that adheres to specification of the corresponding format version that is defined in the primary header (§2.2).

§2.2 Primary header

The primary header is 16 bytes long and includes information about the mod format itself. All offsets are calculated starting from the end of the signature (0x8). The following fields are placed directly after the signature:

Table 2.2.1 — Generic header fields

Offset, HEX	Type	Comment
0x0	u16	Format version, minor. The change in this value indicates a minor change in the format that does not break compatibility with tools made for the previous version of the format. Must be 3 according to the current specification.
0x2	u16	Format version, major. The change in this value indicates a breaking change in the format that introduces a difference significant enough to require an updated toolset. Must be 0 according to the current specification. The order is swapped, making the direct version value comparison possible by reading it as an u32 value (little endian).
0x4	u32	Number of header chunks (refer to §2.3).
0x8	u64	File size. This includes everything from the first byte of the signature to the last byte of the file. 8 bytes long because I am afraid of Reggyxt’s mods.
0x10	u8	Alignment byte. If 0 (default), all alignments and padding are ignored, the data is stored as is. If 1, alignment applies as stated in corresponding sections.
0x11	u8[7]	Reserved. Must be 0.

§2.3 Header chunks

Everything that is stored in the file is considered a chunk. There are mandatory, optional and custom chunks.

— Mandatory chunks must be present in every mod file. If a required chunk is missing, the mod is considered malformed.

— Optional chunks are supposed to exist, but the mod is not considered malformed if they are missing. They are generally used to provide additional information to the user.

— Custom chunks are implementation dependent zones which can be utilised by the developers of modding tools for various purposes. It is required that custom chunks clearly indicate their purpose and size, or the tool developer must provide a public documentation for their use cases to avoid storage of ambiguous and/or malicious data inside mod files. All tools working with custom chunks must clearly label them as such in any preferred way that is not hidden from the end user.

Every chunk is defined with a u32 chunk ID (not necessarily readable ASCII text, but not null) and u32 chunk data size (not including the chunk ID and the size bytes themselves). The rest of the data inside the chunk depends on the chunk type. Chunks are 32-bit aligned (unless the alignment is off). If the chunk data prevents the alignment, null bytes are put after the data to even it out (padding bytes). These bytes are not included in the chunk size field.

Table 2.3.1 —Required and optional header chunks

FourCC	**Name**	**Data**	**Required (Type)**
meta	Metadata	u64 offsets from the beginning of the file to the metadata chunk (§2.3.1) of every mod. The metadata is not stored in the chunk table directly in order to avoid chunk ID collisions. If there are multiple mod files in a single file, this is calculated relative to the beginning of this particular mod.	Yes
tool	Tool	A UTF-8 encoded string (no BOM) stating tool name and version. If several tools were used, they are separated by a null byte. All strings are null-terminated, including the last one if there are multiple. *Example:* ‘`IAMT v1.2.3`’ The difference between this field and the corresponding field in metadata chunk is that this field defines the primary tool that was used to make a mod. The field in metadata may be used if several mods are being combined and some of them were created using a different tool.	Yes (0) Optional (1)
entr	Entries	u32 entries amount followed by u64 offset to the first entry.	Yes
cksm	Checksum	If the data size value is null (or the field is completely missing), no data validation is used and the following fields are omitted. Otherwise, the value is interpreted as the chunk data size, followed by the data itself: A u32 value for string length in bytes (includes null terminator) followed by a UTF-8 encoded string (no BOM) that defines the hashing/verification algorithm. A u32 value for hash length followed by the binary	Optional

FourCC	Name	Data	Required (Type)
		representation of the hash itself, or other data necessary for validation. Verification type values: 'CRC32/CKSUM'[1], 'CRC32/POSIX', 'CKSUM', followed by the u32 hash length of 4 and the CRC32 of the entire file excluding the hash itself. When the hash is calculated, the place dedicated for it is represented with zeros.	
type	Mod file type	A u8 value that defines the mod type: • 0 — regular mod file; • 1 — a part of the other Type 0 mod. Optionally, if chunk data size is > 1, the following values are added: A u8 value: • if Type 0: the total number of parts for a given mod. This does not include the Type 0 mod itself, so if you have one Type 0 and one Type 1 file, this value is '1'; • if Type 1: number of the current part, starting from 1. A u16 value — the length of a mod group identifier, followed by the identifier itself (a null-terminated UTF-8 string). Type 1 mods are considered parts of Type 0 mods as long as their identifiers match. It is better not to expose this field to the end user to avoid situations where several mods have similar identifiers. A short randomly generated string should work fine. Optional values can only be omitted if the mod part is in the same file as the base mod, there is only one base mod in the file and it is the first one, making the rest of the included files parts automatically. If this field is missing, the mod is automatically considered Type 0.	Optional

§2.3.1 Metadata chunk

The metadata chunk defines information about every mod stored in the file. Each entry starts with a chunk ID followed by the data. All entries are 32 bits aligned. The order is not strict.

The first 4 bytes of the metadata chunk define the number of metadata entries. The rest is filled with metadata entries. If custom entries are introduced, the value after entry ID must be the entry size to let other tools skip the area.

[1] Names taken from https://reveng.sourceforge.io/crc-catalogue/17plus.htm#crc.cat-bits.32.

Table 2.3.1.1 — Metadata entries

Chunk ID	Name	Data	Required (Type)
auth	Author	A u32 value for string length in bytes (includes null terminator) followed by a UTF-8 encoded string (no BOM).	Yes (0) Optional (1)
mnam	Mod name	A u32 value for string length in bytes (includes null terminator) followed by a UTF-8 encoded string (no BOM). Must not allow new lines (‘\n’) due to §4. Type 1: this field defines the name of the mod part (‘weapon skins’, ‘custom music 1’ and so on).	Yes
desc	Description	A u32 value for string length in bytes (includes null terminator) followed by a UTF-8 encoded string (no BOM).	Optional
mver	Mod version	A u32 value for string length in bytes (includes null terminator) followed by a UTF-8 encoded string (no BOM). Must not allow new lines (‘\n’) due to §4.	Yes (0) Optional (1)
game	Game version	A u32 value for string length in bytes (includes null terminator) followed by a UTF-8 encoded string (no BOM). If several games are compatible, they are separated by a semicolon (‘;’). *Example:* ‘`Chicken Invaders Universe v173.1`’, *for several games* ‘`CIU v140;CI1R v2`’.	Optional
tool	Tool	A u32 value for chunk data length in bytes (includes null terminator) followed by a UTF-8 encoded string (no BOM) stating tool name and version. If several tools were used, they are separated by a semicolon (‘;’). If the field is missing, the one from the primary header is used by default.	Optional
date	Creation date	An s64 time_t POSIX time, UTC.	Optional[2]
pswd	Password protection	A u32 value. If the value is null or this entry is missing, anyone can extract the mod file and the following fields are omitted. Otherwise, the value is interpreted as the length of the data that follows it: A u32 value for string length in bytes (includes null terminator) followed by a UTF-8 encoded string (no BOM) that defines the hashing/verification algorithm. A u32 value for hash length followed by the binary representation of the hash itself. If a different password confirmation way to comparing the password hash is used, this field can store other data. Hash type values: ‘CRC32/CKSUM’[3], ‘CRC32/POSIX’, ‘CKSUM’, followed by the u32 hash length of 4 and the CRC32 of the password, which is required to match the	Optional

[2] Although not required, it’s generally assumed that this timestamp is created automatically by the toolset using the user’s local time.

[3] Names taken from https://reveng.sourceforge.io/crc-catalogue/17plus.htm#crc.cat-bits.32.

Chunk ID	Name	Data	Required (Type)
		CRC32 of the one received from the user. Note that this is not encrypting the data of the mod file itself, as it still needs to be somehow installed without entering any kind of password, so this is merely a soft prevention of generic mod repacking by other users.	
icon	Mod icon	4 bytes of ANSI text (‘\0’ is a null byte) for format definition if it cannot be recognised automatically. Case sensitive, lowercase. Only PNG and 32-bit TGA are required to be supported for compressed and not compressed data respectively. The rest are optional. Possible values: ‘png\0’, ‘tga\0’, ‘bmp\0’, ‘jpg\0’, ‘jxl\0’, ‘webp’, and so on. null — leave the recognition to the file header. A u32 value for icon file size. Icon file data follows. The proposed format to support is a square 256x256 PNG file but as long as the file can be recognised reliably it can be of any format or dimensions necessary.	Optional
trgt	Target	A u32 value for string length in bytes followed by a UTF-8 encoded string (no BOM). Stores the intended file name to mod, e.g. ‘CIU.dat.222x.easter’. It should not be enforced, but this target defines the defaults for a given mod file. If it ends with ‘.mp3’, it’s assumed that this mod is intended for mobile version of the game. Multiple values are separated by a semicolon (‘;’).	Optional

§2.4 Entries

Every file that is replaced, added or removed by a mod is defined with an entry. Every entry is 32 bits aligned. Null bytes are added after the string for padding. The WAD 4 djb2 file path hash is not stored because it can be generated from the path.

Table 2.4.1 — Entry structure

Offset, HEX	Type	Data
0x0	u64	File offset (relative to the start of the current mod file)
0x8	u64	File size
0x9	u8	Mod installation file treatment bitmask, low to high: bit 0 — add (0) the WAD entry or skip (1) if it’s missing. bit 1 — replace (0) the WAD entry or skip (1) if it exists. If both bits are 1, the file is removed from WAD archive if it

Offset, HEX	Type	Data
		exists and skipped if missing. In this case file offset and file size fields are not being read and must be set to 0. Bits 2–3 are reserved for future format revisions. Bits 4–7 are for user defined flags. If the flag is useful, suggest adding it as the standard flag for the next format revision. IAMT behaviour would be 0b00000001 (skip if the WAD entry is missing, replace if it exists). Default value for most mods would be the same. For mods which add new files (e.g., in seasonal WAD archives) 0b00000000 can be used. If the tool creates an 'uninstaller' mod with original files, 0b00000011 can be used to remove such files which were not originally present.
0xA	u8 ('strlen')	String length (includes null terminator)
0xB	UTF-8 string, null terminated.	File path to replace. Uses forward slashes, doesn't have quotation marks. *Example:* '`music/theme-CI3.ogg`'

§2.5 Combining several mods

The format does not explicitly define special areas for mod files with several groups of mods. The only requirement is that the start of the next mod file is aligned by 4 bytes. However, mod files can include mods for several WAD archives at once by using Type 1 mods in the same file as the Type 0 mod. The same strategy can be used for mods which include several different mods which may or may not be enabled/selected by user.

§2.6 Backward compatibility with legacy formats

UVEMT (IAMT) will be providing a conversion utility for CIUMod files when the new format is adopted by it.

§3 FILE PATHS TABLE FORMAT

§3.1 UVEPaths signature

The first 8 bytes of an UVE file table always contain the following byte array:

Listing 3.1.1 — UVE file table header signature

```
55 56 45 50 61 74 68 73
```

This translates to an ASCII sequence '`UVEPaths`', or a hexadecimal u64 value of `0x7368746150455655 (8316024673246991957)`. If the signature matches, the app should be able to safely assume that this is a compatible table format that adheres to specification of the corresponding format version. The file extension is '.uvepaths'.

§3.2 Primary header

The primary header is 12 bytes long and includes information about the table. All offsets are calculated starting from the end of the signature (0x8).

Table 3.2.1 — Generic header fields

Offset, HEX	Type	Comment
0x0	u16	Format version, minor. The change in this value indicates a minor change in the format that does not break compatibility with tools made for the previous version of the format. Must be 4 according to the current specification.
0x2	u16	Format version, major. The change in this value indicates a breaking change in the format that introduces a difference significant enough to require an updated toolset. Must be 0 according to the current specification. The order is swapped, making the direct version value comparison possible by reading it as an u32 value (little endian).
0x4	u32	Number of header chunks (refer to §3.3).
0x8	u64	File size. This includes everything from the first byte of the signature to the last byte of the file.
0x10	u8	Alignment byte. If 0 (default), all alignments and padding are ignored, the data is stored as is. If 1, alignment applies as stated in corresponding sections.
0x11	u8[7]	Reserved. Must be 0.

§3.3 Header chunks

Similar to §2.3.

Table 3.3.1 —Required and optional header chunks

Chunk ID	Name	Data	Required (Type)
meta	Metadata	u64 offsets from the beginning of the file to the metadata chunk (§3.3.1). The metadata is not stored in the chunk table directly in order to avoid chunk ID collisions. If there are	Optional

Chunk ID	Name	Data	Required (Type)
		multiple mod files in a single file, this is calculated relative to the beginning of this particular mod.	
tool	Tool	A UTF-8 encoded string (no BOM) stating tool name and version. If several tools were used, they are separated by a null byte. All strings are null-terminated, including the last one if there are multiple. *Example:* `'IAMT v1.2.3'`	Yes
entr	Entries	u32 entries amount followed by u64 offset to the first entry.	Yes
cksm	Checksum	If the data size value is null (or the field is completely missing), no data validation is used and the following fields are omitted. Otherwise, the value is interpreted as the chunk data size, followed by the data itself: A u32 value for string length in bytes (includes null terminator) followed by a UTF-8 encoded string (no BOM) that defines the hashing/verification algorithm. A u32 value for hash length followed by the binary representation of the hash itself, or other data necessary for validation. Verification type values: 'CRC32/CKSUM'[4], 'CRC32/POSIX', 'CKSUM', followed by the u32 hash length of 4 and the CRC32 of the entire file excluding the hash itself. When the hash is calculated, the place dedicated for it is represented with zeros.	Optional

§3.3.1 Metadata chunk

The metadata chunk defines information about the file. Each entry starts with a chunk ID followed by the data. All entries are 32 bits aligned. The order is not strict.

The first 4 bytes of the metadata chunk define the number of metadata entries. The rest is filled with metadata entries. If custom entries are introduced, the value after entry ID must be the entry size to let other tools skip the area.

Table 3.3.1.1 — Metadata entries

FourCC	Name	Data	Required (Type)
game	Game version	A u32 value for string length in bytes (includes null terminator) followed by a UTF-8 encoded string (no BOM). If several games are compatible, they are separated by a semicolon (';'). *Example:* `'Chicken Invaders Universe v173.1'`, *for several games* `'CIU v140;CI1R v2'`.	Optional

[4] Names taken from https://reveng.sourceforge.io/crc-catalogue/17plus.htm#crc.cat-bits.32.

FourCC	Name	Data	Required (Type)
date	Creation date	An s64 time_t POSIX time, UTC.	Optional[5]

§3.4 Entries

Every entry is 32 bits aligned (unless the alignment is off). Null bytes are added after the file path string for padding.

Table 3.3.1 — Table entry structure

Offset, HEX	Type	Data
0x0	u8	String length (includes null terminator)
0x1	UTF-8 string, null terminated.	File path. Uses forward slashes. *Example:* '`music/theme-CI3.ogg`'

§3.5 Backward compatibility with legacy formats

Compared to IAMT table format this does not store hashes, as now we know that djb2 is used. Instead, we can get them at runtime. The file size should be reduced as now we don't waste space to fill all 0x5C bytes.

[5] Although not required, it's generally assumed that this timestamp is created automatically by the toolset using the user's local time.

§4 INSTALLED MODS DATA (“INSTDATA”)

A text file in the same directory as WAD file called ‘mods.txt’, UTF-8. The first not commented or empty row always contains two numbers separated by a comma: major and minor version of the format in this order. Every game archive defines a separate section using its name in single square brackets. Inside every section the data is listed in the following order, separated by new lines (‘\n’):

[WAD file name]

[[Mod file name]]

[[[0:1]]] — ‘part number:total parts’. Type 0 always has 0:0. Type 0 with 2 parts has 0:2 and parts themselves have 1:2 and 2:2 respectively.

Name=Mod name from its metadata.

Version=Mod version.

Created=ISO 8601 mod creation date string. A single dash (‘-’) if missing.

Installed=ISO 8601 date string. A single dash (‘-’) if missing.

File name 1 — name of the replaced file.

File name 2...

[[[1:1]]]

Name=Mod part name

File name 1...

...and so on.

An example file is shown below. Rows which start with a hash (‘#’) or semicolon (‘;’) are ignored and treated as comments. Two hashes (‘##’) can be used as a start for some tool-specific custom information which is treated as a regular comment by other tools.

Although not required, it is recommended to parse the file after mod installation and remove the entries which are being overwritten by the new mod so there are no duplicates. If duplicates exist, the last occurrence takes the priority.

Listing 4.1 — Example file

```
0.2
[CIU.dat.222x.standard]
[[EpicMusicMod.uvemod]]
[[[0:0]]]
Name=Epic music mod
Version=v1.3
Created=2025-10-19T15:37:01Z
Installed=2025-10-19T15:42:37Z
music/quiet.ogg

[CIU.dat.222x.easter]
[[appl.uvemod]]
[[[0:0]]]
Name=Apple to fumo
Version=v1.0
Created=2024-02-11T10:48:29Z
Installed=2025-10-19T15:43:02Z
gfx/arcade/appleCore.tga
gfx/arcade/appleCore.tga.ini.preload
```

§5 FILE EXAMPLE

An example file is provided with this specification along with ImHex pattern for UVEMod format. It contains one Type 0 mod with 3 entries and one Type 1 mod part with 2 entries. Type 0 also contains a custom ‘cmpt’ chunk to illustrate how they are handled.

§6 CONTRIBUTORS

The current revision of the document was created with contribution from the following people:

— Dawid8
— Jade
— ravenDS
— VerMishelb

§7 CHANGES BETWEEN REVISIONS

2026-04-05 Rev 0.4

The document revision and the format versions may not be the same from now on. As of Rev 0.4, installed mods info described in §4 is not finalized, as it requires extensive testing with real mods in order to find an optimal way of storing this information.

§2.3.1

Clarified that the only required formats to be supported are PNG and TGA. The rest are optional and support is up to developers of different toolsets.

§3

Added a checksum field to UVEPaths to make it possible to check for table corruptions.

Reworked the UVEPaths format structure to make it possible to extend it later. It is now similar to UVEMod.

2026-03-02 Rev 0.3

§2.2

Added the alignment field. 0 disables the alignment. Any other value aligns areas which mention padding by 4 bytes relative to the start of the current mod file.

§2.3.1

'icon': format identifier and the icon file size were swapped.

§2.4

Clarified that file pointers are calculated relative to the start of the current mod file that's processed, not relative to the beginning of the file as a file system element.

2026-03-02 Rev 0.2

§1.1

Corrected terminology.

§2.1

File format identifiers no longer scream at you (UVEMOD → UVEMod).

§2.3

The 'trgt' .mp3 paragraph was previously cut abruptly, which is now corrected.

Added integrity check field ('cksm').

Added file type field ('type') to make combined mods possible (e.g., two mod files, in a single file or not, one of which is for CIU.dat.222x.standard and the other is for CIU.dat.222x.xmas).

§2.3.1

Changed the structure of metadata chunk: now the first value there is the amount of metadata entries.

‘tool’, ‘trgt’ are now semicolon separated for consistency with ‘game’. Was separated by a null byte in revision 0.1.

‘pswd’: reworked the field to allow different kinds of a hash in case this is ever needed for some reason.

‘icon’: ‘ciu\0’ is not an allowed format anymore because there is no reason for it to be used in place of the icon, unless the game is going to display it which it isn’t. You can still use it as if it was a custom format if you really want to.

§2.4

Reordered entries to make them easier to read in a program. Should also help with speed due to better alignment.

Added §2.5 Combining several mods.

§3

Rebranded UVETABLE to UVEPaths to make the purpose clearer.

§4

Reworked the file structure to support mod parts, reduce space consumption, useless repetitions and increase readability in plain text.

Added semicolons as possible comment sign.

2025-10-19 Rev 0.1

First revision.